

LOW-LEVEL DESIGN

For

Ai Based Phishing Detection For Retail

ABHAY KRISHNA P

Register No: 23BCCED001

Course: Bachelor of Computer Applications (BCA)

Yenepoya Institute of Arts, Science, Management & Commerce

Balmatta

Date: 23/04/2026

Under the guidance of

Mr vikyath shetty

Lecturer, Department of Computer Science

Yenepoya Institute of Arts, Science, Management & Commerce

Kulur

Submitted to



YENEPOYA

(DEEMED TO BE UNIVERSITY)

Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND
MANAGEMENT**

Balmatta, Mangalore

Yenepoya (Deemed to be University)

Section No.	Title	Page No.
1	Introduction	1
1.1	Purpose of Document	1
1.2	Scope of the System	2
1.3	Definitions & Abbreviations	3
2	System Overview	4
2.1	System Description	5
2.2	Architecture Overview	6
2.3	Technology Stack	8
3	Module Design	9
4	Database Design	11
5.1	Phishing Detection APIs	12
6	Security Considerations	14
8	Future Enhancements	16
9	Testing Strategy	17
9.1	Unit Testing	17
9.2	Integration Testing	17
9.3	Functional Testing	17
9.4	Performance Testing	18
9.5	Error Handling Testing	18
9.6	User Acceptance Testing	18
10	Conclusion	19

1 Introduction

The AI-Based Phishing Detection System is designed to identify and classify emails as phishing or legitimate using machine learning techniques through a unified and intelligent platform. The system supports the analysis of multiple input types, including text, images, and real-time email data, enabling comprehensive detection of potential security threats. This High-Level Design (HLD) document presents the overall system architecture, key components, and interactions between different modules, focusing on the structural design and data flow rather than implementation-level details. The system follows a modular and scalable architecture that ensures efficient processing and allows easy integration of additional features in the future. By combining web technologies, OCR, email services, and machine learning models, the system aims to deliver accurate, fast, and user-friendly phishing detection while enhancing overall cybersecurity awareness.

1.1 Purpose of Document

This document presents the Low-Level Design (LLD) of the AI-Based Phishing Detection System. It outlines the internal architecture, module-level structure, data flow, and interactions between system components required to build an efficient and secure phishing detection solution.

The purpose of this document is to:

- Define the system modules and their detailed components

- Explain the data processing flow for text, image, and email inputs
- Provide a clear technical design for development
- Ensure secure handling of user inputs and email data
- Describe the integration of machine learning and OCR modules

This LLD serves as a blueprint for developers to implement the system effectively while maintaining high standards of performance, accuracy, and security.

1.2 Scope of the System

The scope of the AI-Based Phishing Detection System in this Low-Level Design includes the detailed implementation of modules responsible for processing and classifying email content using machine learning techniques. The system handles multiple input types such as text, image-based inputs through OCR, and real-time email data via IMAP. It defines the internal working of components including input handling, preprocessing, feature extraction, model prediction, and result generation. The scope also covers secure handling of user inputs, temporary data management, and efficient interaction between frontend and backend modules. The system is designed for accurate and fast phishing detection in a web-based environment, with a modular structure that supports future enhancements and integration of additional features.

1.3 Definitions & Abbreviations

This section explains important terms and abbreviations used in the AI-Based Phishing Detection System to maintain clarity and consistency throughout the document.

❖ Definitions

➤ Phishing Detection

The process of identifying and classifying emails as phishing or legitimate by analyzing their content using machine learning techniques.

➤ Machine Learning (ML)

A computational approach that enables the system to learn patterns from data and make predictions or classifications without explicit programming.

➤ Optical Character Recognition (OCR)

A technique used to extract textual information from images, allowing the system to analyze image-based phishing content.

➤ Email Monitoring

A process that retrieves and analyzes emails from a mail server in real time to detect potential phishing attempts.

➤ Preprocessing

The method of cleaning and transforming input data into a suitable format before applying machine learning algorithms.

➤ Feature Extraction

The process of converting textual data into numerical

representations that can be used by the machine learning model.

➤ Prediction

The output generated by the system indicating whether an email is phishing or legitimate.

➤ Frontend Interface

The user-facing part of the system where users provide input and view results.

➤ Backend Processing

The server-side operations responsible for handling requests, processing data, and generating results.

➤ API Communication

The interaction between frontend and backend components through defined endpoints for processing and returning data.

❖ Abbreviations

Abbreviation	Full Form
ML	Machine Learning
AI	Artificial Intelligence
OCR	Optical Character Recognition
IMAP	Internet Message Access Protocol
API	Application Programming Interface
UI	User Interface
HTTP	HyperText Transfer Protocol
NB	Naive Bayes
CSV	Comma-Separated Values

2 System Overview

The system design of the AI-Based Phishing Detection System explains the internal working of the application, including its components, processing flow, and interaction between modules. At the Low-Level Design (LLD) stage, the focus is on how each function operates and how data moves through different parts of the system. The application is built using the Flask framework, which handles user requests and manages communication between modules. The design follows a modular structure where each component performs a specific task, improving maintainability and scalability. The system accepts different types of input such as user-entered text, uploaded images, and real-time emails retrieved from Gmail using the IMAP protocol. For image inputs, an OCR module using pytesseract and OpenCV is used to extract text for further analysis.

2.3 System Description

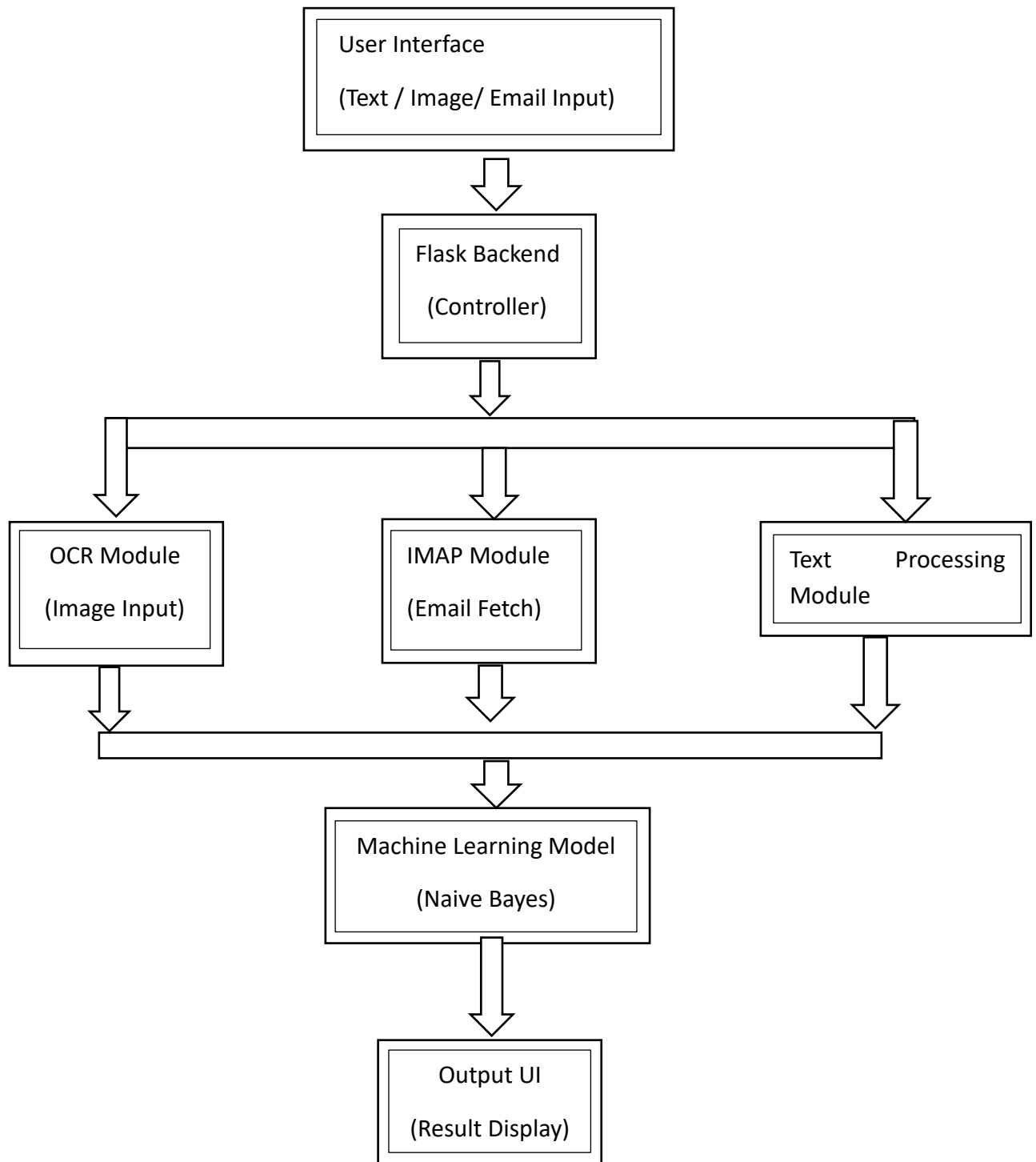
The AI-Based Phishing Detection System is a web-based application designed to analyze and classify email content as phishing or legitimate using machine learning techniques. The system processes multiple input types, including user-entered text, uploaded images, and real-time emails fetched using the IMAP protocol. It is built using the Flask framework, which handles user requests and coordinates between different modules. The application follows a modular structure where components such as input handling, OCR processing, feature extraction, and machine learning classification work together to generate results. Image inputs

are processed using OCR to extract text, which is then analyzed by the model. The system provides fast and accurate results through a simple user interface, ensuring efficient phishing detection and smooth interaction between frontend and backend components.

2.3 Architecture Overview

The AI-Based Phishing Detection System follows a modular client-server architecture that enables efficient interaction between the user interface and backend components. The frontend provides a user-friendly web interface for entering text, uploading images, and monitoring emails. The Flask backend handles incoming requests and coordinates communication between different modules. The system includes key components such as input processing, OCR for extracting text from images, email monitoring using IMAP, and a machine learning model for classification. Each module performs a specific task to ensure smooth and accurate data flow throughout the system. The application processes inputs in real time and generates results with minimal delay. It is designed to handle multiple user requests efficiently without affecting performance. The architecture is scalable and maintainable, allowing easy integration of additional features and future improvements. The system also ensures secure handling of user inputs and temporary data during processing. It supports multiple input formats, increasing flexibility and usability. Error handling mechanisms are implemented to maintain system stability during failures.

The modular design allows independent updates and maintenance of each component. The system can be extended to support advanced models and additional security features. Overall, the architecture provides a reliable and efficient framework for phishing detection.



2.2 Architecture

2.3 Technology Stack

The AI-Based Phishing Detection System is developed using a combination of machine learning techniques, image processing tools, and web technologies to ensure accurate and efficient phishing detection. The system integrates text processing, image-based analysis using OCR, and real-time email handling to provide a comprehensive solution.

- Python – Core programming language used for backend development and system logic
- Flask – Lightweight web framework used to handle requests and manage application flow
- Scikit-learn – Used to implement machine learning algorithms such as Logistic Regression for email classification
- TF-IDF Vectorizer – Converts text data into numerical features for model training and prediction
- Pandas & NumPy – Used for dataset handling and preprocessing operations
- Tesseract OCR (pytesseract) – Extracts text from image-based phishing content
- OpenCV (cv2) – Performs image preprocessing to improve OCR accuracy
- Pillow (PIL) – Handles image loading and processing
- IMAP Protocol – Fetches real-time emails from Gmail for analysis

- HTML & CSS – Used to design the frontend user interface

3 Module design

The system is divided into multiple modules, each responsible for a specific function. This modular design improves maintainability, scalability, and ease of development. Each module works independently and interacts with others to perform phishing detection efficiently.

- ❖ User Interface Module

This module provides the frontend interface where users interact with the system. It allows users to enter email text, upload images, and access email monitoring features. The interface is designed to be simple and user-friendly, ensuring easy navigation and input handling.

- ❖ Input Handling Module

This module processes user inputs received from the interface. It handles different input types such as text data, uploaded image files, and email credentials. The module ensures that the input is properly validated and forwarded to the appropriate processing modules.

- ❖ OCR Module (Image Processing)

This module extracts text from uploaded images using Tesseract OCR and OpenCV. It converts image-based phishing content into readable text, which can then be analyzed by the machine learning model.

- ❖ Email Monitoring Module

This module connects to the email server using the IMAP protocol. It retrieves recent emails from the user's inbox and extracts relevant content such as subject and body for analysis. The fetched data is then passed to the detection module.

❖ Preprocessing Module

This module cleans and prepares the input data before analysis. It removes unwanted characters, normalizes text, and converts it into a suitable format for feature extraction.

❖ Feature Extraction Module

This module uses TF-IDF vectorization to convert textual data into numerical features. These features are required by the machine learning model for classification.

❖ Machine Learning Module

This module uses a Logistic Regression model to classify the input data as phishing or legitimate. It analyzes patterns in the data and generates predictions along with confidence scores.

❖ Detection Module

This module integrates the preprocessing, feature extraction, and machine learning components to produce the final result. It determines whether the input is phishing or safe and prepares the output for display.

❖ Output Module

This module displays the results to the user through the interface. It shows the classification result along with confidence level and relevant messages.

❖ API Module

This module handles communication between frontend and backend using REST APIs. It processes requests and sends responses in structured format (HTML/JSON).

4 Database Design

The AI-Based Phishing Detection System does not use a traditional database in its current implementation and instead relies on file-based storage and in-memory processing. The dataset used for training the machine learning model is stored in a CSV file, while the trained model and vectorizer are saved as serialized files. During execution, all data processing is performed in memory without permanent storage. Uploaded images are temporarily stored for OCR processing and are not retained after use, and email data fetched through IMAP is analyzed in real time without being stored. This lightweight approach reduces system complexity and improves performance. In future, a database system such as MySQL or MongoDB can be integrated to store logs, results, and user data for better scalability and management.

5 API Specifications

The AI-Based Phishing Detection System uses RESTful APIs to enable communication between the frontend and backend components. These APIs are responsible for handling user inputs such as text, image uploads, and real-time email data, and for coordinating the processing of this information within the system. The backend manages the flow of data, applies OCR for image-based inputs, and utilizes machine learning techniques to perform phishing detection. The system is designed to ensure efficient processing, secure data handling, and accurate result generation.

5.1 Phishing Detection APIs

Email Detection (Text/Image Input)

POST

Processes user input (text or image) and performs phishing detection.

Request Body (Form Data):

```
json
{
  "email_text": "Your account is suspended. Click
here to verify.",
  "image": "uploaded_file.png"
}
```

Response:

```
json
{
  "result": "Spam",
  "confidence": 0.92,
  "reason": "Detected as phishing/spam"
}
```

Status Codes:

200: Detection successful

400: Invalid input

500: Processing error

2. Email Monitoring (Real-Time Analysis)

POST /monitor_real

Fetches recent emails from the user's inbox and performs phishing detection.

Request Body:

```
json
{
  "email": "user@gmail.com",
  "password": "app_password"
}
```

Response:

```
json
{
  "emails": [
    {
      "text": "Your order is confirmed",
      "result": "Not Spam",
      "confidence": 0.85
    },
    {
      "text": "Urgent! Verify your account",
      "result": "Spam",
      "confidence": 0.95
    }
  ]
}
```

```
]
}
```

Status Codes

200: Emails fetched and analyzed successfully

401: Authentication failed

500: Server or connection error

6 Security Considerations

The AI-Based Phishing Detection System incorporates essential security measures to ensure safe handling of user inputs and data during processing. All inputs, including text, images, and email credentials, are validated and sanitized to prevent malicious content and common web vulnerabilities. Email monitoring is performed using secure IMAP over SSL to ensure encrypted communication with mail servers. The system avoids permanent storage of sensitive data, and any temporary data such as uploaded images or email content is cleared after processing. Error handling mechanisms are implemented to prevent exposure of internal system details. Additionally, the system design minimizes session usage and ensures that any temporary access data is securely managed. These measures collectively help maintain data integrity, confidentiality, and overall system reliability.

7 Error Handling & Validation

The system implements error handling and validation mechanisms to ensure stable operation and accurate processing of user inputs. These mechanisms help prevent system crashes, handle

unexpected conditions, and maintain data integrity during execution.

➤ Input Validation

All user inputs such as text, image files, and email credentials are validated before processing. This ensures that the data is in the correct format and prevents invalid or malicious inputs from affecting the system.

➤ Exception Handling

The system uses exception handling techniques to manage runtime errors during operations such as OCR processing, email fetching, and model prediction. Errors are captured and handled without interrupting the system flow.

➤ File Handling Errors

Uploaded files are checked for valid formats and proper upload before processing. Invalid or corrupted files are rejected to prevent processing failures.

➤ API Error Handling

The system ensures proper handling of API requests and responses. In case of errors, appropriate messages are returned to the user without exposing internal system details.

➤ System Stability

Error handling mechanisms ensure that the system continues to function smoothly even when unexpected inputs or failures occur.

8 Future Enhancements

The system can be further improved by integrating advanced technologies and expanding its capabilities to enhance accuracy, performance, and scalability.

➤ Advanced Machine Learning Models

Future versions can incorporate deep learning techniques to improve detection accuracy and handle more complex phishing patterns.

➤ Database Integration

A database can be added to store user inputs, results, and logs for better data management and analysis.

➤ Improved OCR Accuracy

Enhancing image preprocessing techniques can improve text extraction from complex or low-quality images.

➤ Cloud Deployment

Deploying the system on cloud platforms can improve accessibility, scalability, and performance.

➤ Real-Time Alerts

The system can be enhanced to provide instant notifications when phishing emails are detected.

➤ Scalability Enhancements

The architecture can be extended to support a large number of users and high-volume data processing.

9 Testing Strategy

The AI-Based Phishing Detection System follows a structured testing strategy to ensure accuracy, reliability, and performance. Different testing methods are applied to validate individual components, overall system functionality, and real-time behavior under various input conditions.

9.1 Unit Testing

Unit testing is performed to verify the functionality of individual modules such as OCR processing, machine learning prediction, and API handling. Each component is tested independently to ensure correct output for given inputs.

9.2 Integration Testing

Integration testing ensures that different modules of the system work together correctly. It verifies the interaction between frontend, backend, OCR module, and machine learning model to confirm smooth data flow.

9.3 Functional Testing

Functional testing validates the system behavior using different types of inputs such as text, image uploads, and real-time email data. It ensures that the system correctly identifies phishing and legitimate emails.

9.4 Performance Testing

Performance testing evaluates the system's response time and efficiency when handling multiple user requests. It ensures that the system provides fast results without performance degradation.

9.5 Error Handling Testing

This testing checks how the system behaves under invalid inputs, missing data, or processing failures. It ensures that errors are handled gracefully without crashing the application.

9.6 User Acceptance Testing

User acceptance testing is performed to ensure that the system meets user expectations. It focuses on usability, accuracy of results, and overall user experience.

10 Conclusion

The AI-Based Phishing Detection System successfully demonstrates an effective approach to identifying and classifying phishing emails using machine learning techniques. The system integrates multiple components such as text analysis, image processing through OCR, and real-time email monitoring to provide comprehensive detection capabilities. The modular design ensures efficient processing, easy maintenance, and flexibility for future enhancements. By combining web technologies with intelligent data analysis, the system delivers accurate and fast results through a user-friendly interface. Overall, the system meets its objectives of improving cybersecurity awareness and assisting users in detecting potential phishing threats in a reliable and efficient manner.